

SeuGado — Kauê: plano de hoje

Desbloquear a equipe e entregar satélite, clima, SAFER e estado projetado

26/09/2026

Meta do dia: terminar tudo aquilo de que os outros quatro dependem, para que amanhã eles só implementem, e entregar a sua parte (satélite, clima, SAFER e estado projetado) funcionando de verdade contra o banco real.

Ordem: primeiro o que desbloqueia a equipe (bloco A), depois o sensoriamento (bloco B), por último o teste integrado e o envio dos documentos (blocos C e D). Se o dia apertar, o bloco A é o que não pode faltar: com ele pronto, a equipe trabalha amanhã com as fixtures mesmo que o seu bloco B termine de madrugada.

1. O que foi decidido hoje

Tudo abaixo já está registrado em `docs/12-REGISTRO-DE-DECISOES-ADR.md` (ADRs 022, 023 e 024) e os números novos já estão no `docs/05-PARAMETROS-CULTIVARES.md`.

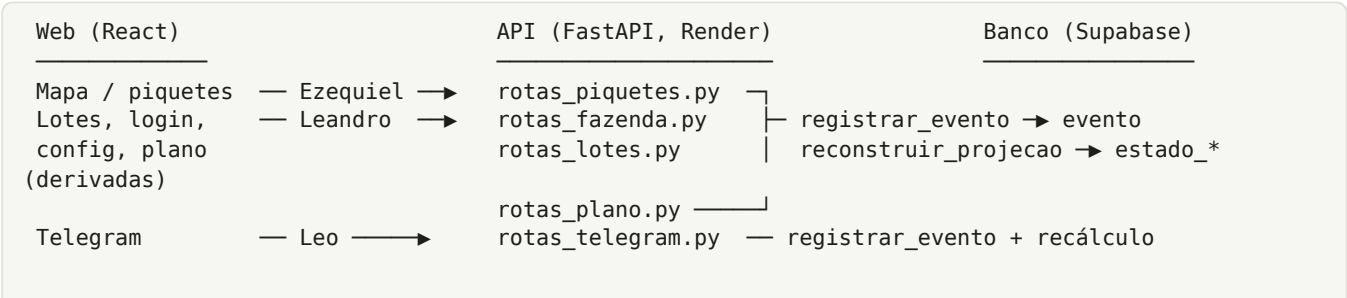
Decisão	Em uma frase	ADR
Divisão da equipe	Cada pessoa é dona de arquivos específicos; ninguém edita arquivo de outro	022
Contratos fixos	<code>src/seugado/contratos.py</code> + <code>frontend/src/lib/tipos.ts</code> + duas fixtures JSON são a “API entre pessoas”	022
Escrita por evento	Cadastro e respostas do bot viram evento → <code>reconstruir_projecao</code> → <code>commit</code> ; as telas leem as tabelas derivadas	022
Fazenda é configuração	A tabela <code>fazenda</code> (funcionários, dias preferenciais, Telegram) é editada por <code>UPDATE</code> , não por evento	022
Infra	Supabase (banco + login) · Render (API) · Vercel (site) · GitHub Actions (ciclo semanal) · Telegram por webhook	022
Âncora do estoque	O estoque de capim parte da altura medida com régua ; o satélite e o clima dão só a taxa de crescimento	023
Evento novo	<code>altura_medida</code> (cadastro, bot ou web)	023
Leitura de satélite	Guarda a observação (NDVI, vermelho, infravermelho, nuvem, pixels), não massa	023
Fontes de dados	HLS no Earth Engine + Open-Meteo (histórico, previsão de 16 dias, ET _o pronta)	023
Validação no MVP	O bot pede altura medida com régua ; a foto fica para depois	023
Ciclo semanal	Segunda 05:00, movimentações só em dia preferencial, recálculo quando o produtor diverge	024
Otimizador	Guloso semanal, simulado dia a dia; fallback: piquete a $\geq 90\%$ da altura de entrada, senão alerta	024
Distância	Haversine entre centroides, em Python (a tabela PostGIS volta com o CP-SAT)	024
Cultivar do MVP	Só o Marandu está completo; as outras aparecem como “aguardando parâmetro”	024

2. Pesquisas feitas hoje

Parâmetro	Valor	Fonte	Confiança
RUE (eficiência de uso da radiação) de <i>B. brizantha</i>	2,31 g/MJ	Almeida et al. (2023), <i>Remote Sensing</i> 15(3):815, SAFER aplicado à Piatã	baixa para Marandu (mesma espécie, outra cultivar)
Temperatura base do Marandu	15,0 °C	Mendonça, Rassini & Villa Nova (2005), Embrapa Pecuária Sudeste	média
Temperatura base do Tanzânia	15,0 °C	idem	média
Temperatura base da <i>B. decumbens</i>	16,7 °C	idem	média
Marandu rotacionado: entrada / saída	30 / 15 cm	Andrade (2008), Embrapa Acre, via Soares et al. (2021)	média
Albedo no SAFER	$\alpha = 0,08 + 0,41 \cdot \rho_{\text{vermelho}} + 0,14 \cdot \rho_{\text{NIR}}$	Teixeira et al. (Embrapa), bandas vermelho/NIR	—
Radiação de onda longa (Slob)	$a_L = 6,99 \cdot T_a - 39,93$	Teixeira et al. (2010, 2012), via Ramos (UNIVASF)	—
Temperatura no ETf	em °C (em kelvin o ETf dá ≈ 0)	verificação numérica da equação	—
Radiação no topo da atmosfera	FAO-56, eq. 21	Allen et al. (1998)	—
HLS no Earth Engine	HLSS30 atualizado até 24/09/2026; HLSL30 parado em 15/10/2025	catálogo do GEE	—
Clima	Open-Meteo: previsão com <code>past_days=92</code> e <code>forecast_days=16</code> ; histórico ERA5 com atraso de ~5 dias; ET ₀ FAO-56 pronta	documentação Open-Meteo	—

Com isso, o Marandu fica com todos os parâmetros: densidade 110, eficiência 0,72, RUE 2,31, temperatura base 15, entrada/saída 30/15 e contínuo 35/20. As demais cultivares não têm densidade nem RUE com fonte. Elas entram no catálogo e aparecem no mapa, mas sem estimativa. **Na demo, use Marandu.**

3. Como os módulos se conectam



Ciclo semanal (jobs/ciclo.py – Leandro), toda segunda 05:00 e sob demanda:

1. confirmar_por_omissao(...)	Leo	delivery/confirmacao.py	
2. ingerir_leituras(...)	KAUÊ	sensing/ingestao.py	(Earth Engine)
3. reconstruir_projecao(...)	KAUÊ	persistencia/projecao_db.py	
4. montar_estado_projetado(...)	KAUÊ	planner/carga.py	(Open-Meteo + SAFER)
5. gerar_plano(estado, fazenda)	João	planner/otimizador.py	
6. salvar_plano(conn, plano)	João	persistencia/planos.py	
7. enviar_plano(conn, plano)	Leo	delivery/envio.py	(Telegram)

Os tipos que atravessam as setas estão em `src/seugado/contratos.py` (SPEC-009):

`EstadoProjetado` sai de você e entra no João; `PlanoManejo` sai do João e entra no Leo, no Leandro (página do plano) e no Ezequiel (cores do mapa).

Arquivos por dono (ninguém edita o arquivo de outro)

Dono	Arquivos
Kauê	db/migrations/*, core/*, persistencia/eventos.py, persistencia/projecao_db.py, persistencia/catalogo.py, contratos.py, sensing/*, planner/estado.py, planner/carga.py, api/main.py, api/deps.py, api/auth.py, pyproject.toml, uv.lock, frontend/package.json, frontend/src/lib/*, frontend/src/componentes/*, frontend/src/App.tsx, tests/fixtures/*
Ezequiel	api/rotas_piquetes.py, cadastro/piquetes.py, frontend/src/paginas/Mapa.tsx, frontend/src/componentes/mapa/*, tests/cadastro/test_piquetes.py
João	planner/otimizador.py, planner/confianca.py, planner/geo.py, persistencia/planos.py, tests/planner/test_otimizador.py, tests/planner/test_confianca.py, tests/persistencia/test_planos.py
Leandro	api/rotas_fazenda.py, api/rotas_lotes.py, api/rotas_plano.py, cadastro/fazenda.py, cadastro/lotes.py, jobs/ciclo.py, .github/workflows/*, render.yaml, frontend/src/paginas/{Login,Onboarding,Configuracoes,Lotes,Plano}.tsx, frontend/src/estilo/*, tests/cadastro/test_fazenda.py, tests/cadastro/test_lotes.py, tests/jobs/*
Leo	delivery/*, api/rotas_telegram.py, scripts/telegram_polling.py, tests/delivery/*

Se alguém precisar de dependência nova ou mudar contrato, pede a você. Você decide aqui comigo.

4. Passo a passo de hoje

Bloco 0 — Contas e chaves (manual, ~40 min)

0.1 Supabase (banco + login)

1. Em `supabase.com`, crie o projeto `seugado` na região **South America (São Paulo)**. Anote a senha do banco.
2. Em *Project Settings* → *Data API*, copie a **Project URL** (`SUPABASE_URL`) e a chave **anon / publishable** (`SUPABASE_ANON_KEY`).
3. Em *Connect*, copie a string do **Session pooler** (porta 5432). Ela é o `DATABASE_URL`. Não use a *Direct connection*: no plano grátis ela é só IPv6, e o Render e o GitHub Actions não alcançam. Não use o *Transaction pooler* (6543): ele quebra os prepared statements do `psycopg`.

4. Em *Authentication* → *Sign In / Providers* → *Email*, desligue **Confirm email**. Na demo, ninguém vai conseguir esperar e-mail de confirmação.

0.2 Google Earth Engine (satélite)

1. Em `console.cloud.google.com`, crie o projeto `seugado-gee` (anote o **Project ID**: ele é o `SEUGADO_GEE_PROJECT`).
2. Em `code.earthengine.google.com/register`, registre esse projeto para **uso não comercial** (acadêmico). A aprovação costuma ser imediata.
3. Ative a **Earth Engine API** no projeto (*APIs e serviços* → *Biblioteca*).
4. Em *IAM* → *Contas de serviço*, crie `seugado-ingestao`, dê os papéis **Earth Engine Resource Viewer** e **Service Usage Consumer**, e gere uma **chave JSON**. O conteúdo do arquivo é o `SEUGADO_GEE_SERVICE_ACCOUNT_JSON`.

0.3 Arquivo `.env local` (fora do Git; o `.gitignore` já ignora `.env`; confira)

Preencha `DATABASE_URL`, `SEUGADO_TEST_DATABASE_URL` (pode ser o mesmo banco hoje), `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SEUGADO_GEE_PROJECT` e `SEUGADO_GEE_SERVICE_ACCOUNT_JSON`. As do Telegram ficam para o Leo.

Bloco A — Desbloquear a equipe (obrigatório hoje)

Cada spec já está pronta em `specs/`, com o kit de aceite em `revisoes/`. O ciclo é o de sempre (`docs/08 §7`).

A1. SPEC-007: eventos do MVP. Adiciona `OrigemPeso`, o evento `altura_medida`, geometria no piquete, leitura de satélite como observação, divergência que move o lote e `combinar_confianca`. Ação manual: Abra o Muse Code e peça para implementar `specs/SPEC-007-eventos-mvp.md`. Quando ele terminar, chame o Antigravity para testar com `revisoes/KIT-ACEITE-007.md` e `commitar`. Avise o Antigravity de que os testes de conformidade 005 e 006 precisam ser ajustados ao novo formato de `leitura_satelite`. É mudança intencional (ADR-023), não regressão.

A2. SPEC-008: banco do MVP. Migração `0002_mvp.sql` (fazenda completa, usuário↔fazenda, catálogo com 9 cultivares, derivadas completas, `plano`, `telegram_conversa`, RLS), `reconstruir_projecao` e o catálogo com overrides. Ação manual: Abra o Muse Code e peça para implementar `specs/SPEC-008-banco-mvp.md`. Depois peça ao Antigravity para aplicar `0001` e `0002` no Supabase (SQL Editor), testar com `revisoes/KIT-ACEITE-008.md` e `commitar`.

A3. SPEC-009: contratos. `contratos.py`, com os tipos e a conversão para JSON. As duas fixtures já estão em `tests/fixtures/` e a spec exige que elas carreguem. Ação manual: Abra o Muse Code e peça para implementar `specs/SPEC-009-contratos.md`. Quando ele terminar, chame o Antigravity para testar com `revisoes/KIT-ACEITE-009.md` e `commitar`.

A4. SPEC-010: esqueleto da API e do site. Antes do Muse, o Antigravity cria o frontend e instala as dependências. Ação manual: Peça ao Antigravity para criar `frontend/` com o template `Vite "react-ts"`, instalar `react-router`, `@supabase/supabase-js`, `leaflet`, `react-leaflet`, `@geoman-io/leaflet-geoman-free` e `@types/leaflet` (dev), e adicionar ao `pyproject fastapi`, `uvicorn[standard]`, `httpx` e `earthengine-api`, com override do `mypy ignore_missing_imports` para o módulo `"ee"`. Ação manual: Depois, abra o Muse Code e peça para implementar `specs/SPEC-010-esqueleto-api-frontend.md`. Por fim, o Antigravity testa com `revisoes/KIT-ACEITE-010.md` e `commita`.

A5. Conferência do bloco A. A API sobe e `GET /saude` responde. `npm run build` passa. O banco tem 9 cultivares. `tests/fixtures/*.json` carregam em `contratos.py`. Deste ponto em diante, **ninguém da equipe fica bloqueado por você**.

Bloco B — Sua implementação

B1. SPEC-011: clima (Open-Meteo). Radiação no topo da atmosfera, graus-dia e cliente de histórico e previsão. Ação manual: Abra o Muse Code e peça para implementar `specs/SPEC-011-clima.md`. Quando ele terminar, chame o Antigravity para testar com `revisoes/KIT-ACEITE-011.md` e `commitar`.

B2. SPEC-012: SAFER. A cadeia de 11 equações, pura, com as faixas de sanidade. Ação manual: Abra o Muse Code e peça para implementar `specs/SPEC-012-safer.md`. Quando ele terminar, chame o Antigravity para testar com `revisoes/KIT-ACEITE-012.md` e `commitar`.

B3. SPEC-013: satélite HLS. Amostragem por piquete no Earth Engine e ingestão idempotente. Ação manual: Abra o Muse Code e peça para implementar `specs/SPEC-013-satellite-hls.md`. Quando ele terminar, chame o Antigravity para testar com `revisoes/KIT-ACEITE-013.md` e `commitar`.

B4. SPEC-014: estado projetado. Estoque de hoje e crescimento dos próximos 14 dias. Ação manual: Abra o Muse Code e peça para implementar `specs/SPEC-014-estado-projetado.md`. Quando ele terminar, chame o Antigravity para testar com `revisoes/KIT-ACEITE-014.md` e `commitar`.

Bloco C — Teste integrado da sua parte

O objetivo é ver, no banco real, o estado projetado de uma fazenda de verdade.

- Desenhe 2 ou 3 piquetes de uma área de pasto real (pode ser em geojson.io) e guarde os polígonos.
- Peça ao Antigravity um script descartável em `scripts/` que:
 - cria uma fazenda de teste e o vínculo com o seu usuário do Supabase;
 - registra `piquete_criado` (Marandu, rotacionado, com a geometria);
 - registra `altura_medida` de cada piquete;
 - roda `reconstruir_projecao` e `commit`;
 - roda `ingerir_leituras` dos últimos 30 dias e `commit`;
 - roda `reconstruir_projecao` de novo;
 - imprime `estado_para_dict(montar_estado_projetado(conn, fazenda_id, hoje))`.
- Critério de pronto:** cada piquete sai com `massa_hoje_kg_ms_ha`, 14 taxas entre 0 e 150 kg/ha/dia e `faltantes` vazio. Se aparecer `estimativa_invalida`, a mensagem de erro do SAFER diz qual variável saiu da faixa. Traga a mensagem aqui num chat [TRIAGEM].
- Guarde o id da fazenda de teste.** O João e o Leo vão usar esse estado real na segunda.

Bloco D — Entregar os documentos

- Os documentos de cada pessoa já estão em `docs/equipe/` (md e pdf). Eles descrevem tudo como disponível, porque você só vai enviar depois de terminar os blocos A a C.
- Mande as chaves **por mensagem privada, nunca pelo Git**:
 - Ezequiel, João e Leandro: `DATABASE_URL`, `SUPABASE_URL` e `SUPABASE_ANON_KEY`.

- Leandro: também as do Earth Engine, porque ele configura o deploy e o GitHub Actions.
- Leo: `DATABASE_URL` (ele cria o próprio bot e o token).

3. Mande a cada um o próprio PDF e o caminho do md no repositório.

5. Checklist antes de enviar

- [] SPEC-007 a SPEC-010 commitadas; `uv run pytest verde`
- [] Migrações 0001 e 0002 aplicadas no Supabase; `SELECT count(*) FROM cultivar = 9`
- [] `uv run uvicorn seugado.api.main:app --app-dir src` sobe; `/saude` responde
- [] `cd frontend && npm run build` passa
- [] SPEC-011 a SPEC-014 commitadas
- [] Teste integrado do bloco C com massa e taxas plausíveis
- [] `git push` feito; branch `main` atualizada
- [] Chaves enviadas em privado

6. Segunda e terça

- **Segunda (reunião):** cada um abre um PR da sua branch; a ordem de merge é Ezequiel → Leandro (lotes/fazenda) → João → Leo → Leandro (ciclo e deploy). Depois, rode o ciclo completo com a fazenda de teste e o bot de verdade.
- **Terça (demo):** mostre o mapa com as cores do plano, a página “Plano da semana” e o Telegram recebendo o plano. Depois, responda “fiz diferente” e mostre o plano recalculado.

Se algo do satélite ou do clima falhar na segunda, o `[TRIAGEM]` é aqui, com o erro e a linha `Leia: docs/11, specs/SPEC-0NN`, o arquivo citado.